

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 10%

QUESTIONS: 15

Total Marks:20

Annexure A

Comparative Analysis

Code of Conduct:

*I **Bandaru Jayasri-192424301** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Criteria	Conceptual Understanding	Accuracy of Answers	Application of Knowledge	Completeness of Response	Clarity & Presentation	Total
Weightage	30%	15%	20%	20%	15%	
Q1						
Q2						
Q3						

Signature of the student:

B.Jayasri

Total Marks :

Annexure A

Short Answer Test

1. Compare **Linear Regression and Polynomial Regression** for predicting housing prices. Analyze differences in model complexity, ability to capture non-linear relationships, risk of overfitting, and interpretability. Discuss scenarios where each model performs better.
2. Compare **Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM)** networks for time-series prediction tasks such as stock price forecasting. Analyze differences in handling long-term dependencies, vanishing gradient problem, computational complexity, and prediction accuracy.
3. Compare **Naïve Bayes and Support Vector Machines (SVM)** for classification problems such as sentiment analysis. Analyze differences in assumptions, decision boundaries, computational efficiency, and performance on high-dimensional data. Discuss scenarios where each method is preferable.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	15	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1) Introduction

Linear Regression and Polynomial Regression are supervised learning algorithms used to predict continuous values such as housing prices.

Linear Regression

Linear Regression assumes a linear relationship between input variables and the target.

$$[\\ y=b_0+b_1x \\]$$

For housing prices, features such as area, number of bedrooms, and age of the house can be used to predict price.

Polynomial Regression

Polynomial Regression extends linear regression by adding powers of the input variables.

$$[\\ y=b_0+b_1x+b_2x^2+\cdots+b_nx^n \\]$$

It can capture curved and nonlinear relationships.

Comparison

Factor	Linear Regression	Polynomial Regression
Relationship	Linear	Nonlinear/curved
Model complexity	Low	Higher
Features	Original features	Includes polynomial terms
Overfitting	Lower risk	Higher risk with high degree
Interpretability	Easy	More difficult
Training	Fast	Relatively slower
Flexibility	Limited	More flexible
Suitable for	Simple relationships	Complex relationships

Housing Price Example

Suppose housing price increases approximately at a constant rate with house area.

A linear model is suitable:

$$[\\ \text{Price}=b_0+b_1(\text{Area}) \\]$$

However, if price increases slowly for small houses and rapidly for larger houses, the relationship may be curved. Polynomial Regression can capture this:

```
[  
Price=b_0+b_1(Area)+b_2(Area^2)  
]
```

Overfitting

Polynomial Regression becomes more complex as the degree increases.

For example:

- Degree 1 → Linear model
- Degree 2 → Quadratic model
- Degree 3 → Cubic model
- Very high degree → Higher overfitting risk

A very complex model may fit the training data extremely well but perform poorly on unseen houses.

When to Use

Linear Regression is preferable when:

- The relationship is approximately linear.
- The dataset is small.
- Interpretability is important.
- A simple and fast model is required.

Polynomial Regression is preferable when:

- The relationship is nonlinear.
- A curved pattern exists in the data.
- Linear Regression underfits the data.
- Sufficient data is available to control overfitting.

Conclusion

Linear Regression is simple, fast and highly interpretable but cannot capture complex nonlinear patterns. Polynomial Regression is more flexible and can model nonlinear relationships, but increasing polynomial degree increases complexity and overfitting risk. Therefore, the model should be selected based on the relationship present in the housing data.

2) Introduction

Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks are neural network architectures used for sequential and time-series data.

For example, they can be used to predict stock prices using historical price and trading information.

RNN

An RNN processes sequential data one step at a time and maintains a hidden state containing information from previous time steps.

It is useful for learning short-term dependencies.

LSTM

LSTM is an improved form of RNN designed to learn long-term dependencies.

It uses three main gates:

- Forget Gate
- Input Gate
- Output Gate

These gates control what information should be remembered or removed.

Comparison

Factor	RNN	LSTM
Architecture	Simple	More complex
Short-term dependency	Good	Excellent
Long-term dependency	Poorer	Better
Vanishing gradient	More affected	Greatly reduced
Training speed	Faster	Slower
Memory	Basic hidden state	Cell state + hidden state
Parameters	Fewer	More
Computational cost	Lower	Higher
Accuracy on long sequences	Often lower	Usually better

Vanishing Gradient Problem

During training, gradients are propagated through many time steps.

In a standard RNN, gradients can become extremely small. This is called the **vanishing gradient problem**.

As a result, the RNN may forget information from much earlier time steps.

LSTM reduces this problem using its cell state and gating mechanism.

Stock Price Prediction

Stock prices depend on both recent and historical information.

For example:

Previous prices
↓
Market trends
↓
Historical patterns
↓
Current information
↓
Predicted price

If long-term patterns are important, LSTM is generally more suitable than a basic RNN.

Computational Complexity

RNNs have fewer parameters and are generally faster to train.

LSTMs have additional gates and parameters, making them more computationally expensive.

However, the additional complexity can improve performance when long-term dependencies are important.

Prediction Accuracy

For short and simple sequences, RNN may provide sufficient accuracy.

For long and complex sequences, LSTM often performs better because it can preserve important information for longer periods.

Conclusion

RNN is simpler and computationally efficient but suffers from difficulties in learning long-term dependencies. LSTM addresses these limitations using memory cells and gates. For complex time-series prediction such as stock forecasting, LSTM is generally preferred when long-term dependencies are important.

3) Introduction

Naïve Bayes and Support Vector Machine (SVM) are supervised machine learning algorithms used for classification. Both can be applied to sentiment analysis to classify text as:

- Positive
- Negative
- Neutral

Naïve Bayes

Naïve Bayes is a probabilistic classifier based on Bayes' theorem.

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

It assumes that features are conditionally independent given the class.

For sentiment analysis, words can be used as features.

For example:

"I love this movie"
↓
Positive probability
↓
Positive sentiment

SVM

SVM finds an optimal decision boundary that separates different classes.

For binary classification, the decision boundary can be represented as:

$$w^T x + b = 0$$

SVM attempts to maximize the margin between classes.

Comparison

Factor	Naïve Bayes	SVM
Type	Probabilistic classifier	Margin-based classifier
Main assumption	Conditional independence	Maximum margin
Output	Class probabilities	Class labels/scores
Decision boundary	Probabilistic	Maximum-margin hyperplane
Training speed	Very fast	Generally slower
High-dimensional data	Excellent	Excellent
Text classification	Very effective	Very effective
Small datasets	Often good	Can work well
Interpretability	Relatively simple	More difficult
Memory	Low	Can be higher

High-Dimensional Data

Text datasets usually contain thousands of features because every unique word can become a feature.

Naïve Bayes handles high-dimensional sparse text efficiently.

SVM can also perform very well on high-dimensional text data because it focuses on finding a separating boundary rather than explicitly modeling every probability.

Decision Boundary

Naïve Bayes determines the class using posterior probabilities.

SVM finds a decision boundary with maximum margin.

For example:

Positive samples	Decision Boundary	Negative samples
+ + +		- - -
+ + +		- - -

The SVM boundary is selected to maximize the separation between the classes.

Computational Efficiency

Naïve Bayes is usually faster to train and predict, making it suitable for large text datasets and real-time applications.

SVM may require more computational resources, particularly with complex kernels and very large datasets.

When to Use Naïve Bayes

Naïve Bayes is preferable when:

- A fast classifier is required.
- The dataset contains high-dimensional text.
- Training data is large.
- Probabilistic predictions are useful.
- A simple model is preferred.

When to Use SVM

SVM is preferable when:

- Classification boundaries are complex.
- High classification accuracy is important.
- The dataset is high-dimensional.
- Sufficient computational resources are available.
- A strong separating boundary is required.

Conclusion

Naïve Bayes and SVM are both effective for sentiment analysis. Naïve Bayes is fast, simple and probabilistic, while SVM focuses on finding the maximum-margin decision boundary and can provide excellent classification performance. For very large text datasets where speed is important, Naïve Bayes is a strong choice. When achieving strong separation between classes is the priority, SVM can be preferable.